

Informe técnico — Primera entrega TPI Food Store

Resumen de lo actuado en Unidades 1, 2 y 3 · verificación en PostgreSQL 18 · IA: OpenCode (+Kiro no disponible)

Unidad 1 — Modelo de datos e integridad

- **ER** completo del dominio Food Store desde el enunciado (5 entidades) con reglas R1–R7 → 01_mer_relacional.md.
- **Paso a relacional**: 1:N por FK, N:M por entidad asociativa (clave sustituta + UNIQUE) → 01_mer_relacional.md.
- **Normalización** de la planilla real hasta 3FN/BCNF con dependencias funcionales explícitas → 02_normalizacion.md.
- **DDL** en schema.sql/ddl.sql: ENUM, IDENTITY, PK/FK con ON DELETE RESTRICT, CHECK, UNIQUE, índices, TIMESTAMPTZ, NUMERIC.

Cómo se probó: schema.sql ejecutado de punta a punta sin errores sobre PostgreSQL 18.

```
food_store_tp3=> \i ddl.sql

CREATE TYPE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
(5 tablas creadas sin errores)
```

Unidad 1/2 — Integridad, transacciones y concurrencia

- Triggers/funciones plpgsql: baja lógica anti-DELETE, descuento de stock, validación de subtotal → triggers_restricciones.sql.
- Laboratorio con 2 sesiones: lectura no repetible, lectura fantasma, bloqueo FOR UPDATE, deadlock → 04_transacciones.md.

Salida real del laboratorio de aislamiento:

```

food_store_tp3=> (sesion A)

BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT stock FROM producto WHERE nombre='Producto 1';
  stock
  -----
     10

-- B: UPDATE stock=50; COMMIT

SELECT stock FROM producto WHERE nombre='Producto 1';
  stock
  -----
     50      <-- lectura no repetible (B commiteo en el medio)
ROLLBACK;

```

Resultado: la sesión B de FOR UPDATE quedó bloqueada ~3,4 s hasta el ROLLBACK de A. Todo cerrado con ROLLBACK (esquema intacto, protocolo de la cátedra).

Unidad 2 — Optimización de consultas

Qué se implementó: consultas de reporting reales (4 tablas), medición con EXPLAIN ANALYZE, reescritura con agregación anticipada e índice de cobertura; se descartaron propuestas que no mejoraban → consultas.sql, TP3/TP4.

Consulta	Antes	Después	Ganancia
C1 facturación por categoría/mes	4387 ms	2052 ms	2,1×
C2 ranking de clientes por gasto	754 ms	751 ms	sin mejora (índice descartado)

Equivalencias SQL verificadas con EXCEPT = 0 filas (RANK vs. subconsulta correlacionada; correlacionada vs. GROUP BY).

Unidad 3 — Índices, vistas y objetos programables

Qué se implementó: 3 índices justificados con medición antes/después, 1 propuesta de IA descartada por sobreindexación, 3 vistas, 3 funciones plpgsql y procedimiento sp_registrar_pedido invocado con CALL; vista materializada de facturación con índice único y REFRESH CONCURRENTLY.

Objeto	Antes	Después	Ganancia
Búsqueda catálogo por nombre	Seq 20 ms	Bitmap 8,4 ms	Seq→Bitmap
Cliente por apellido	Seq 15,9 ms	Index 0,9 ms	~18×
Pedidos tarjeta por mes	78,4 ms	6,8 ms	sin remoción de filas
MV facturación c/mes	2447 ms	~15 ms	~163×

Costo de escritura: 300 INSERT → 47,5 ms (sin índices nuevos) vs 151,6 ms (+3,2×, aceptado).

●●●

```
food_store_tp3=> CALL sp_registrar_pedido(1, ..., ARRAY[1,2], ARRAY[1,1]);

CALL
time: 22 ms -- inserto pedido + 2 detalle, trigger descontó stock
NOTICE: Pedido 200601 registrado para el cliente 1
-- (en BEGIN...ROLLBACK: se revirtió al final de la demo)
```

Uso de IA

- Herramientas: **OpenCode** (IA primaria) y Kiro (requerido, **no instalado**: specs con plantilla Kiro a mano, constado en DUIA).
- SQL de consultas/vistas/índices/triggers/procedimientos → aceptado tras verificación en motor.
- Lectura de plan (TP4 P2): la IA cometió **3 imprecisiones** corregidas con el plan real.
- Índice idx_producto_categoria → **descartado** (redundante).
- Índice pedido(cliente_id, fecha) → **descartado** (sin cambio de plan en la competencia).

Evidencia completa: carpeta Parcial/ + checklist_tpi.md (mapa objetivo → archivo). Reportes TP3/TP4/TP5 en sus carpetas.